THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Software Under Audit: Linux Kernel

Student Name: Rudranarayan Pattnaik
Registration Number: 24BHI10020
Slot: B22
Date of Submission: 29 Mar, 2026

Course: Open Source Software | Unit Coverage: 1 to 5 | Max Marks: 100

Part A: Origin and Philosophy (Units 1 and 2)

A1. The Problem Linux Was Created to Solve

In the late 1980s, the world of operating systems was sharply divided. On one side were commercial Unix systems like SunOS, HP-UX, and AIX, which were powerful but extremely expensive. Universities could sometimes license these, but individual students had almost no way to get their hands on a true Unix-like environment for personal learning and tinkering. On the other side was MINIX, a small Unix-like educational operating system created by Andrew Tanenbaum. MINIX was designed specifically for teaching operating system concepts, and while it ran on cheap PC hardware, it was deliberately limited. Tanenbaum had architected it to remain simple for academic purposes and was not particularly interested in expanding it into a full-featured operating system.

This was the world Linus Benedict Torvalds found himself in when he enrolled at the University of Helsinki in 1988. He was a computer science student with a deep curiosity about how operating systems actually worked at the lowest level. He had access to MINIX through the university, and for a while it served him well enough. But frustration built over time. MINIX lacked a proper terminal emulator. It could not handle tasks that Torvalds wanted to automate. Its design choices were deliberate restrictions, not bugs. Every time Torvalds wanted to go deeper or extend the system, the wall was right there.

In 1991, Torvalds bought himself an Intel 80386 PC, which was a genuinely capable machine at the time. He started writing small programs for it, including a terminal emulator to connect to the university's Unix servers. That terminal emulator grew. He started replacing MINIX components with his own code. He needed a file system, so he wrote one. He needed process handling, so he implemented that too. At some point during the summer of 1991, he realized he was no longer writing a terminal emulator. He was writing an operating system kernel.

On August 25, 1991, Torvalds posted a now-famous message to the comp.os.minix newsgroup. He introduced himself as a Helsinki student and mentioned he was working on a free operating system for 386 and 486 AT clones. He explicitly said it was a hobby, and not something big or professional like GNU. He asked what features people would most want. That message, casual and unassuming, was the public birth of Linux.

The key word in his announcement was free. Torvalds was not thinking about business models or market disruption. He had a real personal need: a Unix-like environment he could control completely and run on hardware he owned. The solution he built happened to be something thousands of other people also needed. That alignment between personal itch and collective need is what gave Linux its remarkable early momentum.

It is also worth noting what Linux was not trying to do. Richard Stallman and the GNU Project had been working since 1983 on a complete free software operating system. By 1991, the GNU project had produced an enormous amount of essential software: compilers, text editors, utilities. What it was still missing was a working kernel. Linux arrived at exactly the right moment to complete that picture. When Torvalds released Linux under the GNU General Public License in 1992, Linux and GNU combined into

what we now call GNU/Linux, a complete free operating system available to anyone willing to download it.

A2. The License: What It Actually Says

The Four Freedoms of Free Software

The Free Software Foundation, founded by Richard Stallman, defines free software through four essential freedoms. Freedom zero is the freedom to run the program for any purpose. Freedom one is the freedom to study how the program works and to change it so it does what you wish. This requires access to the source code. Freedom two is the freedom to redistribute copies to help your neighbour. Freedom three is the freedom to distribute copies of your modified versions to others.

The Linux kernel is licensed under the GNU General Public License version 2. It grants all four of these freedoms. Anyone can download the kernel source code, read it, modify it, run it, and distribute their modified version. The GPL is what keeps these freedoms intact for everyone who receives the software, not just the original recipient.

Can a Company Modify and Sell Linux Without Sharing Changes?

This is where the GPL becomes genuinely interesting. The GPL is a copyleft license, meaning that if you distribute a modified version of Linux, you are required to make the source code of your modifications available under the same GPL license. A company cannot take the Linux kernel, add proprietary improvements, and ship a binary to customers while keeping the source code hidden. If they distribute the binary, they must offer the source.

However, there is an important distinction. If a company runs a modified kernel internally without distributing it to anyone outside the company, they have no obligation to share. Amazon, Google, and countless others run heavily modified versions of Linux in their data centers. They are not distributing those kernels externally, so the GPL's sharing requirement does not trigger. The moment you hand the binary to someone outside your organization, the obligation kicks in. This is sometimes described as the GPL's distribution loophole for cloud computing.

GPL vs MIT: The Key Differences

The MIT license is permissive. It places almost no restrictions on what you can do with the software. You can take MIT-licensed code, modify it, incorporate it into a proprietary product, and ship that product without making your source available. The only thing you must do is preserve the original copyright notice and license text. This makes MIT very attractive to companies that want to build commercial products on open source foundations without being forced to open their own code.

The GPL is restrictive in the sense that it protects downstream freedom. It ensures that software that starts free stays free, no matter who modifies it. The GPL uses copyright law to prevent copyright restrictions from being imposed on the code downstream.

If I were choosing a license for my own project, I would think carefully about my goal. If I want maximum adoption and I am comfortable with companies using my code without giving back, MIT makes sense. If I believe the software should remain free for all users forever, and I want to prevent anyone from creating a proprietary fork, GPL is the right choice. For a kernel or a core piece of infrastructure that society depends on, the GPL philosophy feels more appropriate. For a small utility library that benefits from wide adoption, MIT might serve the project better.

Free as in Beer vs Free as in Freedom

The English word free is genuinely ambiguous, and this ambiguity has caused real confusion in discussions about open source software. Free as in free beer means you get something at no monetary cost. Free as in freedom means you have the right to do what you want with the thing.

Linux is free in both senses, but the freedom sense is what matters philosophically. You can download Ubuntu or Fedora at no cost. That is the beer part. But the more important thing is that you have the right to read every line of the kernel, modify the scheduler, rewrite the memory management system, and distribute your version. A piece of proprietary software might also be free as in beer: plenty of software is available at no cost while keeping the source locked away. What it cannot give you is the freedom to understand or change what it does.

A clear example from Linux is what happened when hardware vendors tried to include proprietary firmware blobs in the kernel tree. The Free Software Foundation and parts of the Linux community objected not to the cost but to the freedom problem: if drivers require closed binary blobs, users lose the ability to understand and modify that part of the system. Distributions like Debian took a strong stance and stripped out non-free firmware. This debate only makes sense if you care about the freedom dimension, not just the price.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The argument for making all software open source is compelling on the surface. Transparency allows security researchers to find and fix vulnerabilities rather than hoping a vendor will notice them. Collective development means that improvements made by one person benefit everyone. Software built on public money, such as government systems or research tools, arguably belongs to the public that funded it. Open source also prevents vendor lock-in: if the company behind your critical software goes bankrupt or changes direction, you are not helpless.

The argument against is also real. Writing good software is expensive work. Open source licenses, especially permissive ones, do not guarantee that the people who use the software will contribute back or fund the developers. The sustainability problem is serious: many critical open source libraries are maintained by a single volunteer who is not being paid for the work. The Heartbleed vulnerability in OpenSSL in 2014 revealed that one of the most widely used security libraries on the internet was maintained by a

tiny team with almost no funding, despite being trusted by billions of users. Making software open does not automatically make it well-maintained. Proprietary models, for all their restrictions, do create funding for sustained engineering work.

My view is that the question is not binary. For infrastructure that society depends on, for operating systems, compilers, cryptography libraries, and web servers, openness is clearly better. For consumer applications where rapid feature development and a specific user experience matter, proprietary development can work well. The ethical imperative shifts depending on how foundational the software is.

Is It Ethical for Companies to Profit from Free Community Labor?

This is one of the genuinely hard questions in the open source world. Companies like Red Hat, Google, and Amazon have built billion-dollar businesses on top of Linux, which was and is developed largely by volunteers and by engineers at smaller companies. Is this parasitic?

The honest answer is that it depends on what the company does next. Red Hat's model was historically considered ethical by much of the community because it employed Linux kernel developers, contributed substantial code back, and built its business around support and services rather than restricting the software itself. When IBM acquired Red Hat in 2019, many in the community watched carefully to see whether this would change. In 2023, Red Hat did restrict access to RHEL source code in a way that upset CentOS users, and the community response was sharp and immediate. Forks like AlmaLinux and Rocky Linux arose specifically to fill the gap.

Google has contributed enormous amounts to the Linux kernel, Android's kernel patches being just one example. But Google also uses Linux to power infrastructure that generates enormous profits, and the contribution does not feel proportional to the extraction. Meta runs some of the world's largest Linux-based infrastructure. Their contributions are real but selective.

I think the ethical line is crossed when a company takes the benefit, contributes nothing back, and actively works against the community when it becomes inconvenient for them. Contributing code, funding developers, or participating in governance is how companies earn their place in the ecosystem. Pure extraction without reinvestment is ethically uncomfortable, even if the GPL technically permits it in certain circumstances.

Standing on the Shoulders of Giants: Enabler or Underminer of Innovation?

The phrase originally referred to building on the accumulated knowledge of those who came before. In software, almost no meaningful work is done without relying on the work of others. The Linux kernel itself is built on decades of Unix research. Python runs on C. Web browsers run on operating systems. Every layer depends on layers below it.

Open source accelerates this stacking dramatically. When a developer can study how the Linux virtual file system works, they learn not just a product but a solution to a general problem. They can apply that thinking to new problems. When they can modify and extend it, they are not starting from zero. The pace of innovation in software since the rise of open source has been genuinely extraordinary, and the availability of reusable building blocks is a major reason why.

The concern is that standing on giants without acknowledging or sustaining them leads to the giants getting tired and sitting down. If the maintainers of critical libraries burn out because no one is paying them, the foundation weakens. Open source enables innovation best when the ecosystem includes mechanisms for sustaining the foundational projects, whether that is corporate sponsorship, foundation funding, or community support. Innovation built entirely on invisible volunteer labor is standing on a foundation that could crack.

Part B: Linux Footprint (Unit 2)

This section documents how the Linux kernel lives on a Linux system, specifically observed on a running installation. The commands and observations below are from an actual Linux environment.

Installation Method

The Linux kernel is the core of the operating system itself, so it is not installed the same way application software is. On most distributions, the kernel arrives as part of the base system installation. On Debian-based systems like Ubuntu, you can install a specific kernel version using apt:

On RPM-based systems like Fedora or CentOS, the equivalent is dnf install kernel or yum install kernel. For those who want to run a custom or bleeding-edge kernel, it must be compiled from source, which involves downloading the source from kernel.org, configuring it with make menuconfig, building it with make, and installing it with make install and make modules_install.

Key Directories

The Linux kernel and its related components occupy several important locations in the file system. The compiled kernel image lives in /boot, along with the initramfs initial RAM disk image and the GRUB bootloader configuration. The kernel modules, which are loadable pieces of the kernel for drivers and functionality that does not need to be compiled in directly, are stored in /lib/modules followed by the kernel version string.

Configuration for running kernel parameters can be found in /proc/sys, and the full kernel configuration used to build the running kernel is usually found at /boot/config followed by the kernel version, or at /proc/config.gz on systems where that option was enabled.

User and Group

The kernel itself does not run as a user in the traditional sense. It operates below the user space entirely. However, user space processes that interact with the kernel through system calls run under specific users and groups. The root user has the highest privilege in user space and can perform operations that affect the kernel's behavior, such as loading modules with insmod or modprobe, or adjusting kernel parameters with sysctl.

The reason user and group separation matters for security is the principle of least privilege. Processes running as non-root users cannot load arbitrary kernel modules, cannot modify most /proc/sys parameters, and cannot access hardware directly. If a non-root process is compromised by an attacker, the damage is limited to what that user can do. This is why web servers run as www-data rather than root, even though the kernel underneath is the same for all of them.

Service Management

The kernel itself is not managed as a systemd service: it is the thing that runs systemd. However, kernel-adjacent functionality is managed through services. The kernel module loading service, the system logging that reads from the kernel ring buffer, and hardware management services are all controlled through systemctl on modern systems.

You can check what kernel modules are currently loaded with lsmod, load a module with modprobe, and remove one with modprobe -r. The kernel ring buffer, which contains the kernel's own log messages from boot and runtime, is accessible with dmesg.

Update Model

Security patches for the Linux kernel come through a layered process. The kernel developers at kernel.org release stable updates, often addressing security vulnerabilities, on a regular schedule. Distribution maintainers like the Ubuntu Security Team or the Red Hat engineering team take those patches, backport them if necessary to their supported kernel version, and push them through the distribution's package update system.

On Ubuntu or Debian, running apt update followed by apt upgrade will pull in kernel security patches as they become available. On Fedora, dnf update handles this. The system does not automatically boot into the new kernel: the old kernel remains on disk and in the GRUB menu until explicitly removed, which gives administrators a fallback if the new kernel introduces a regression.

Part C: The FOSS Ecosystem (Units 3 and 4)

What the Linux Kernel Depends On

The Linux kernel is written predominantly in C, with small amounts of assembly language for architecture-specific low-level code. To compile it, you need GCC or Clang, both of which are open source compilers. The GNU Make build system coordinates the compilation process. GNU Binutils provides the linker and other binary tools used during the build. The kernel's cryptographic subsystems use algorithms from open source cryptographic libraries. The build process itself relies on tools like Python for configuration scripts and OpenSSL for signing kernel modules.

At runtime, the kernel interfaces with a broad collection of user space tools. The GNU C Library, glibc, provides the standard interface between user programs and kernel system calls. Tools like util-linux provide fundamental utilities such as mount, fdisk, and lsblk. Systemd, which is also open source, serves as the init system that the kernel hands control to once booting is complete.

What Linux Has Enabled and Inspired

The list of things Linux has directly enabled is almost impossibly long. Android, which runs on the majority of smartphones in the world, uses the Linux kernel as its base. The servers that power Google, Amazon, Facebook, Netflix, and essentially every major internet service run Linux. Supercomputers: nearly all of the top 500 systems in the world run Linux. Embedded devices from routers to smart TVs often run minimal Linux distributions.

Linux also inspired a culture of open operating system development. The BSDs, FreeBSD, OpenBSD, and NetBSD, are not derived from Linux but developed in parallel, and the success of Linux demonstrated that community-developed operating systems could be serious, production-grade software. Chrome OS is a Linux-based operating system. Steam OS, Valve's gaming platform, is Linux-based. Raspberry Pi OS is a Linux distribution.

Forks and derivatives of Linux itself are numerous. Android is the most significant. Ubuntu, Fedora, Debian, Arch, and hundreds of other distributions are all built around the same Linux kernel but with different philosophies, package managers, and target users. The kernel has maintained remarkable compatibility across all of these because of the stable ABI guarantee: user space programs compiled for Linux continue to work even as the kernel evolves.

The LAMP Stack Connection

Linux sits at the foundation of the classic LAMP stack, which stands for Linux, Apache, MySQL, and PHP. This stack powered a vast proportion of the web from the early 2000s onward. Linux provides the operating system. Apache, covered elsewhere in this course's software options, provides the web server. MySQL provides the database. PHP provides the server-side scripting. All four components are open source, and their combination made it possible for small developers and organizations to build web applications without licensing fees.

Even as the LAMP stack has evolved, with Nginx often replacing Apache, PostgreSQL sometimes replacing MySQL, and Python or Node.js sometimes replacing PHP, Linux has remained constant at the bottom. It is the L that never gets replaced.

Community and Governance

The Linux kernel community is one of the largest and most active open source communities in the world. Development happens through a mailing list, the Linux Kernel Mailing List or LKML, which has been running since the early 1990s and generates thousands of emails per day. Patches are submitted, reviewed, debated, and accepted or rejected through this mailing list and a hierarchy of maintainers.

Linus Torvalds remains the final decision-maker for what goes into the mainline kernel, a structure sometimes called the Benevolent Dictator for Life model. Under him is a layer of subsystem maintainers who manage specific areas of the kernel: networking, memory management, storage, graphics. The Linux Foundation, a non-profit organization, provides organizational support, hosts events like the Linux Plumbers Conference and the Linux Foundation Open Source Summit, and employs some kernel developers directly.

Corporate contributors are enormous in number. According to Linux Foundation reports, companies like Intel, Samsung, Google, Red Hat, and Huawei are consistently among the largest contributors of code to the kernel. This makes Linux unusual: it is simultaneously a community project and something that major corporations depend on deeply enough to pay engineers to improve.

Part D: Open Source vs Proprietary (Critical Analysis)

The closest proprietary alternative to the Linux kernel is the Windows NT kernel, which underpins Windows Server and Windows desktop operating systems. The comparison below is honest and does not assume that open source wins on every dimension.

Dimension	Linux Kernel (Open Source)	Windows Server (Proprietary)
Cost	1400	*1069*
Security (code auditability)	full source code	*proprietary*
Support and reliability	*enterprise*	*comprehensive*
Freedom to modify	*full*	*none*
Community vs corporate control	*full*	*less*
Overall verdict	*better*	*less than linux*

Conclusion and Recommendation

After examining the Linux kernel across its history, license, ecosystem, and technical footprint, the picture that emerges is of software that has genuinely changed the shape of the computing world. This is not marketing language. The internet as it exists today runs almost entirely on Linux. The smartphones in billions of pockets run a Linux kernel underneath Android. The supercomputers advancing climate science and drug discovery run Linux. A student in 1991 built something that scratched his own itch, and it turned out that itch was universal.

For a real-world deployment, Linux is the obvious recommendation for server infrastructure, cloud workloads, embedded systems, and any environment where control, cost, and security auditability matter. Windows Server has genuine advantages in environments that are deeply integrated with Microsoft tooling, Active Directory, and where the support contract model is preferred by IT management. But for the majority of backend infrastructure decisions, the flexibility, the community support, the total cost of ownership, and the transparency of the Linux kernel make it the stronger choice. As for contributing: yes. Any engineer who benefits professionally from Linux has a reasonable obligation to give something back, whether that is code, documentation, bug reports, or financial support for the Linux Foundation.

Shell Script Tasks

Script 1: System Identity Report

This script serves as a welcome screen for the Linux system. It displays the distribution name, kernel version, logged-in user, home directory, system uptime, current date and time, and a message about the open source license governing the operating system. The key concepts used are shell variables, the echo command, command substitution with the $() syntax, and basic output formatting with string padding.

Output Screenshot

```
┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ sed -i 's/\r$//' script1.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ chmod +x script1.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ ./script1.sh
═══════════════════════════
 Open Source Audit
═══════════════════════════
Student: Apurv Gupta
Software: Linux
Distro: Kali GNU/Linux Rolling
Kernel: 6.12.38+kali-amd64
User: kali
Uptime: up 7 minutes
Date: Thu Mar 26 11:16:21 AM EDT 2026
License: GPL/MIT (based on OS)
```

Explanation

The script assigns system information to variables using command substitution: uname -r captures the kernel version, whoami captures the current user, and uptime -p provides a human-readable uptime string. The lsb_release command retrieves the distribution name. The date command formats the current timestamp. Each variable is then echoed inside a formatted block, padded with separator lines for readability. The license message is a static string since the GPL v2 status of the Linux kernel is not something the system reports automatically. This approach demonstrates how shell scripts can combine dynamic system queries with static informational content.

Script 2: FOSS Package Inspector

This script checks whether a specified package is installed on the system, retrieves its version and license information, and uses a case statement to print a one-line philosophy note about the package based on its name. The concepts demonstrated include if-then-else conditionals, the case statement, package manager queries using rpm -qi or dpkg -l, and pipe filtering with grep.

Output Screenshot

```
┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ sed -i 's/\r$//' script2.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ chmod +x script2.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ ./script2.sh
linux-image-6.12.38+kali-amd64 is installed
ii  linux-image-6.12.38+kali-amd64        6.12.38-1kali1                        amd64        Linux 6.12 for 64-bit PCs
No description available

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ sS
```

Explanation

The script first checks whether the target package is installed by running the package manager query command and redirecting both standard output and standard error to /dev/null. If the command returns exit code zero, the package is present and its metadata is retrieved and filtered. If not, the user gets a clear not-installed message. The case statement then matches the package name against known patterns to print a contextual note about that software's place in the open source world. This structure shows how scripts can combine system queries with conditional logic to produce informative, contextual output.

Script 3: Disk and Permission Auditor

This script loops through a predefined list of important system directories and reports the disk usage and permission information for each one. A separate check at the end specifically looks for the Linux kernel configuration directory. The concepts demonstrated are the for loop, array iteration, the if conditional, the du command for disk usage, and ls -ld combined with awk to extract permission fields.

Output Screenshot

```
┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ sed -i 's/\r$//' script3.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ chmod +x script3.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ ./script3.sh
Directory Audit
/etc ⇒ drwxr-xr-x root root | Size: 15M
/var/log ⇒ drwxr-xr-x root root | Size: 47M
/home ⇒ drwxr-xr-x root root | Size: 2.4G
/usr/bin ⇒ drwxr-xr-x root root | Size: 704M
/tmp ⇒ drwxrwxrwt root root | Size: 8.0K
Checking /etc for configs...
adduser.conf
alsa
alternatives
apache2
apparmor

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$
```

Explanation

The script defines a bash array of directory paths and iterates through them using a for loop. For each directory, it checks existence with the -d test operator before attempting to read it, which prevents errors on systems where some directories may not exist. The ls -ld command retrieves the long listing for the directory itself rather than its contents, and awk extracts the permissions, owner, and group fields. The du -sh command provides human-readable disk usage. The additional check for the kernel configuration directory at the end demonstrates how the script connects back to the specific software being audited.

Script 4: Log File Analyzer

This script reads a log file line by line, counts how many lines contain a specified keyword, and prints a summary. It also prints the last five matching lines. The script accepts both the log file path and the search keyword as command-line arguments. The concepts demonstrated are the while-read loop, the if conditional inside a loop, counter

variables using arithmetic expansion, command-line argument handling with $1 and $2, and default values with the := syntax.

Output Screenshot

```
┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ ./new_script4.sh log.log error
Keyword 'error' found 1 times
error
```

Explanation

The script validates that the log file exists before proceeding, exiting with a meaningful error message if it does not. The while IFS= read -r loop reads the file one line at a time without stripping leading whitespace or interpreting backslashes, which is the correct approach for processing arbitrary log content. Inside the loop, grep -iq performs a case-insensitive match. Each match increments a counter using the $(( )) arithmetic expansion syntax. After the loop completes, the final count is printed. The last five matching lines are retrieved by running grep again on the file and piping through tail -5, which is more efficient than storing matches in memory during the loop.

Script 5: The Open Source Manifesto Generator

This script interactively asks the user three questions, then composes a personalized open source philosophy statement using their answers and saves it to a text file. The concepts demonstrated are the read command for user input, string concatenation and variable interpolation in multi-line text, writing output to a file with redirection, the date command for timestamps, and a demonstration of the alias concept through a comment.

Output Screenshot

```
┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ sed -i 's/\r$//' script5.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ chmod +x script5.sh

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$ ./script5.sh
Answer 3 questions
Tool you use: git
Freedom means: control
What you'd build: privacy focused version social platform
Saved to manifesto_kali.txt
On Thu Mar 26 11:27:52 AM EDT 2026, I believe open source means control. I use git daily and would build privacy focused version social platform and share it freely.

┌──(kali㉿kali)-[~/Desktop/Scripts/scripts]
└─$
```

Explanation

The script uses read -p to prompt the user and capture input into named variables. The three pieces of input, an everyday tool, a word for freedom, and something the user would build and share, are then woven into a paragraph using echo with variable interpolation. The heredoc or multi-line echo approach writes the complete paragraph to a file named with the current username. The date command adds a timestamp so the manifesto is dated. A comment near the top of the script demonstrates the alias concept

by showing how a long command could be shortened, connecting the scripting exercise to the course's discussion of shell customization.

References

The following sources were used as starting points for research and background understanding. The written analysis in this report is original and in the author's own words.

- GNU Project: The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative: The Open Source Definition. opensource.org/osd
- Eric S. Raymond: The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds and David Diamond: Just for Fun: The Story of an Accidental Revolutionary. HarperCollins, 2001.
- The Linux Command Line by William Shotts. linuxcommand.org
- GNU Bash Manual. gnu.org/software/bash/manual
- SPDX License List. spdx.org/licenses
- Linux Foundation: 2022 Linux Kernel History Report. linuxfoundation.org
- kernel.org: Official Linux Kernel Archives. kernel.org
- Linus Torvalds original announcement on comp.os.minix, August 25, 1991. Archived at groups.google.com